

Universidad Nacional de San Agustín



Facultad de Ingeniería de Producción y Servicios

Escuela Profesional de Ingeniería de Sistemas

PROGRAMACIÓN DE SISTEMAS

DOCUMENTACIÓN TÉCNICA

Proyecto Final: Programa ADMIN en Linux

Docente:

Norman Patrick Harvey Arce

Integrantes:

Condori Catunta, Joselin Sharon

Rojas Condori, Fabiana Paola

Schreiber Landeo, Diego Hans

2026

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. Equipos, materiales y temas utilizados	2
4. Arquitectura del Sistema	3
5. Estructura del Proyecto	3
6. Descripción de los módulos	4
6.1. Administrador de tareas	5
6.2. Shell de archivos	5
6.3. Ejecución de comandos Linux	5
6.4. Sistema de respaldos	6
6.5. Analizador de scripts Bash	6
6.6. Cola de descargas	6
7. Flujo general del sistema	6
7.1. Diagrama de flujo general	7
8. Llamadas al sistema y funciones utilizadas	8
8.1. Administración de procesos	8
8.2. Administración de archivos	8
8.3. Ejecución de comandos	8
8.4. Entrada y salida	9
8.5. Manejo de directorios	9
8.6. Resumen	9
9. Compilación e instalación del proyecto	9
9.1. Requisitos del sistema	9
9.2. Estructura del proyecto	10
9.3. Proceso de compilación	10
9.4. Limpieza del proyecto	10
9.5. Ejecución del programa	10
10.Pruebas realizadas	10
10.1. Pruebas del administrador de tareas	11
10.2. Pruebas del shell de archivos	11
10.3. Pruebas de ejecución de comandos	11
10.4. Pruebas del sistema de respaldos	12
10.5. Pruebas del analizador Bash	12
10.6. Pruebas de la cola de descargas	12
10.7. Resumen de pruebas	12
11.Conclusiones y trabajos futuros	13
11.1. Conclusiones	13
11.2. Trabajos futuros	13

1. Introducción

El presente proyecto consiste en el desarrollo de una aplicación de consola en lenguaje C para el sistema operativo Linux, que proporciona herramientas de administración del sistema de forma modular e interactiva. El sistema permite gestionar procesos, archivos, comandos personalizados, respaldos, análisis de scripts Bash y colas de descargas.

El propósito técnico es consolidar los conocimientos de programación de sistemas, haciendo uso intensivo de llamadas al sistema (system calls) como `fork()`, `exec()`, `wait()`, `kill()`, `opendir()`, `readdir()`, entre otras, así como el manejo de señales, tuberías y procesos en entorno Unix. La aplicación está estructurada de manera que cada módulo es independiente y puede ser desarrollado y probado por separado, facilitando el trabajo colaborativo mediante Git.

2. Objetivos

2.1. Objetivo General

Desarrollar una herramienta de administración de sistemas Linux en C que integre funcionalidades esenciales para la gestión de procesos, archivos, comandos, respaldos, análisis de scripts y descargas, aplicando los conceptos de programación de sistemas y buenas prácticas de desarrollo.

2.2. Objetivos Específicos

- Implementar un administrador de procesos que permita listar, buscar, finalizar, suspender y reanudar procesos, así como mostrar el árbol de procesos y el consumo de recursos.
- Desarrollar un shell de archivos con operaciones básicas (listar, copiar, mover, eliminar) usando llamadas al sistema.
- Crear un ejecutor de comandos Linux que permita al usuario introducir cualquier comando, mostrar su salida y errores, y guardar un historial.
- Diseñar un sistema de respaldos que comprima y almacene copias de seguridad de directorios.
- Construir un analizador sintáctico para scripts Bash que valide la estructura de los comandos.
- Gestionar una cola de descargas de archivos desde URLs.
- Organizar el proyecto de forma modular con Makefile, documentación y control de versiones.

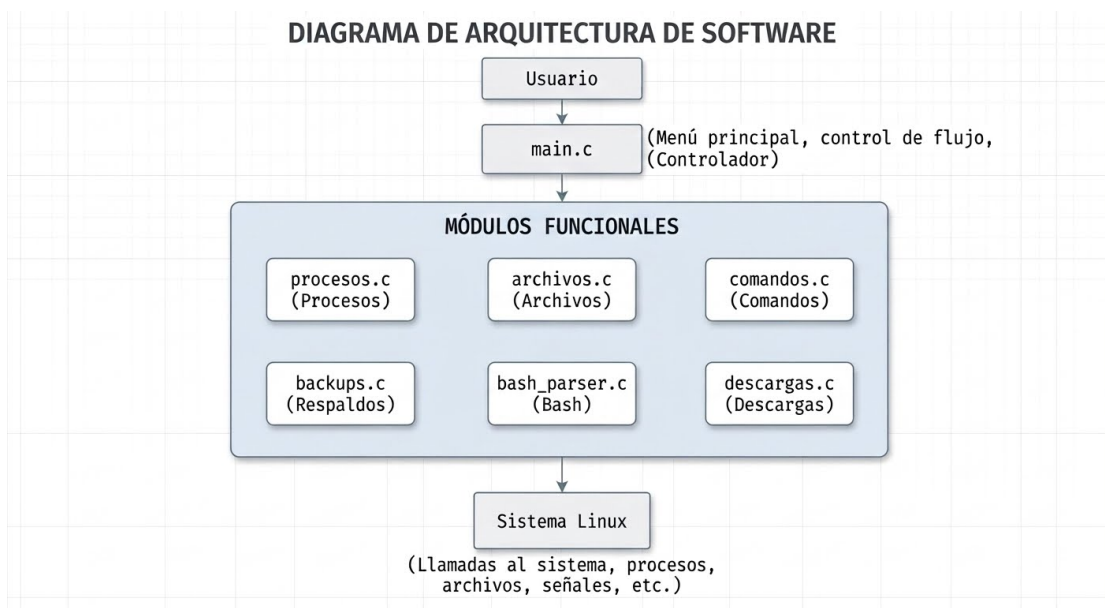
3. Equipos, materiales y temas utilizados

- **Lenguaje C (estándar GNU99)** – Lenguaje de programación principal utilizado para el desarrollo del proyecto, seleccionado por su eficiencia y compatibilidad nativa con las llamadas al sistema operativo.
- **Sistema Operativo Linux Ubuntu 22.04 LTS** – Entorno de ejecución e infraestructura base sobre la cual se despliega y prueba el administrador del sistema.
- **Compilador GCC (v11+)** – Colección de compiladores de GNU utilizada para transformar el código fuente en C en el ejecutable binario final del sistema.
- **Librerías estándar de C** – Cabeceras esenciales (`stdio.h`, `stdlib.h`, `string.h`, `unistd.h`, `signal.h`, `sys/types.h`, `sys/wait.h`, `dirent.h`, `sys/stat.h`, `time.h`) utilizadas para la gestión de entrada/salida, manejo de memoria, control de procesos y manipulación de archivos.

- **Llamadas al sistema (System Calls)** – Funciones del núcleo de Linux (`fork()`, `execvp()`, `execlp()`, `wait()`, `waitpid()`, `kill()`, `opendir()`, `readdir()`, `stat()`) empleadas para interactuar directamente con el sistema operativo a bajo nivel.
- **Make** – Herramienta de automatización utilizada para gestionar la compilación eficiente del proyecto mediante un archivo `Makefile`.
- **Git** – Sistema de control de versiones descentralizado empleado para registrar el historial de cambios y coordinar el desarrollo del código.
- **GitHub** – Plataforma de alojamiento en la nube utilizada para almacenar el repositorio remoto y facilitar el trabajo colaborativo.
- **LaTeX** – Lenguaje de marcado utilizado para la redacción formal de este informe.
- **Overleaf** – Plataforma online colaborativa usada para redactar el informe técnico en LaTeX.

4. Arquitectura del Sistema

La arquitectura del sistema es de tipo monolítica modular, donde el punto de entrada es `main.c`, el cual controla el menú principal y deriva la ejecución a los módulos especializados.



Cada módulo tiene su propio submenú y funciones independientes, lo que permite que diferentes integrantes trabajen en paralelo sin conflictos. La comunicación entre módulos se realiza únicamente a través de `main.c`, que invoca las funciones de menú de cada módulo.

5. Estructura del Proyecto

El árbol de directorios del proyecto es el siguiente:

```

Proyecto-Admin-Linux
|--- Makefile           # Script de compilacion
|--- README.md          # Descripcion general
|--- .gitignore         # Archivos ignorados por Git
|
  
```

```

|--- src                                # Código fuente
|   |--- archivos.c
|   |--- backups.c
|   |--- bash_parser.c
|   |--- comandos.c
|   |--- descargas.c
|   |--- main.c
|   |--- main.o
|   |--- procesos.c
|
|--- include                            # Archivos de cabecera
|   |--- archivos.h
|   |--- backups.h
|   |--- bash_parser.h
|   |--- comandos.h
|   |--- descargas.h
|   |--- procesos.h
|
|--- backups                            # Directorio copias de seguridad
|
|--- scripts                            # Scripts Bash prueba para analizador
|   |--- prueba2.sh
|   |--- prueba3.sh
|
|--- historial                          # Historial de comandos ejecutados
|   |--- historial.txt
|
|--- downloads                          # Archivos descargados
|
|--- docs                               # Documentacion
|   |--- Manual_Usuario.pdf
|   |--- Documentacion_Tecnica.pdf
|   |--- Presentacion.pptx
|
|--- bin                                # Ejecutable compilado
|   |--- admin_linux
|

```

Descripción de cada carpeta:

- **src/**: Contiene todos los archivos .c con la implementación de cada módulo y el main.
- **include/**: Archivos .h con los prototipos de funciones y definiciones compartidas.
- **backups/**: Espacio de almacenamiento para los respaldos generados por el módulo de respaldos.
- **scripts/**: Scripts de prueba en Bash para evaluar el analizador sintáctico.
- **historial/**: Archivo de texto donde se registran los comandos ejecutados con fecha y hora.
- **descargas/**: Destino de los archivos descargados mediante el módulo de descargas.
- **docs/**: Documentación del proyecto en formato PDF y presentación.
- **bin/**: Ejecutable final del proyecto, generado por el Makefile.

6. Descripción de los módulos

El proyecto fue desarrollado utilizando una arquitectura modular, donde cada componente implementa una funcionalidad específica de la herramienta de administración para Linux. Esta organización facilita

el mantenimiento, la reutilización del código y la incorporación de nuevas funcionalidades sin afectar el resto del sistema.

6.1. Administrador de tareas

Este módulo permite supervisar y administrar los procesos activos del sistema operativo. Sus principales funciones son:

- Listado de procesos en ejecución.
- Búsqueda de procesos por nombre.
- Visualización del consumo de CPU y memoria.
- Finalización de procesos mediante señales del sistema.
- Suspensión y reanudación de procesos.
- Visualización del árbol de procesos para identificar relaciones padre-hijo.

La implementación utiliza comandos del sistema y llamadas al sistema de Linux para interactuar con los procesos existentes.

6.2. Shell de archivos

Este módulo proporciona una interfaz sencilla para la administración de archivos y directorios. Entre sus funciones se encuentran:

- Listar archivos de un directorio.
- Copiar archivos.
- Mover archivos.
- Eliminar archivos.
- Buscar archivos por nombre.
- Mostrar estadísticas básicas de archivos y directorios.

Estas operaciones permiten administrar el sistema de archivos sin abandonar la aplicación.

6.3. Ejecución de comandos Linux

Este componente permite ejecutar comandos directamente desde la herramienta. Sus características principales son:

- Recepción del comando ingresado por el usuario.
- Ejecución utilizando el intérprete de comandos.
- Visualización de la salida estándar.
- Registro de los comandos ejecutados en un historial.

El historial permite consultar posteriormente las operaciones realizadas durante la sesión.

6.4. Sistema de respaldos

El módulo de respaldos permite generar copias de seguridad de archivos y directorios importantes. Entre las funciones implementadas destacan:

- Creación de respaldos.
- Almacenamiento organizado de versiones.
- Restauración de respaldos previamente generados.
- Consulta del historial de respaldos disponibles.

Esta funcionalidad reduce el riesgo de pérdida de información durante la administración del sistema.

6.5. Analizador de scripts Bash

Este módulo analiza archivos escritos en Bash con el objetivo de identificar algunos de sus componentes principales.

Actualmente permite detectar:

- Variables declaradas.
- Estructuras repetitivas (for y while).
- Información básica del script analizado.

El análisis se realiza leyendo el archivo línea por línea e identificando patrones característicos del lenguaje Bash.

6.6. Cola de descargas

Este módulo administra una lista de descargas simuladas y registra los eventos asociados.

Entre sus funciones se encuentran:

- Agregar nuevas descargas.
- Mostrar la cola de descargas.
- Registrar eventos durante la ejecución.
- Mostrar el progreso de las tareas.

Este componente demuestra el manejo organizado de tareas pendientes dentro de la aplicación.

7. Flujo general del sistema

La herramienta fue diseñada siguiendo un flujo de ejecución basado en un menú principal desde el cual el usuario puede acceder a los diferentes módulos del sistema. Cada opción seleccionada invoca una función específica encargada de realizar la operación correspondiente y, al finalizar, retorna nuevamente al menú principal.

El flujo de ejecución general puede resumirse de la siguiente manera:

1. Inicio de la aplicación.
2. Carga del menú principal.

3. Selección de una opción por parte del usuario.
4. Ejecución del módulo correspondiente.
5. Procesamiento de la solicitud.
6. Presentación de los resultados.
7. Retorno al menú principal.
8. Finalización del programa cuando el usuario selecciona la opción de salir.

Este diseño permite mantener una interacción sencilla con el usuario y facilita la incorporación de nuevos módulos sin modificar la estructura principal del programa.

7.1. Diagrama de flujo general

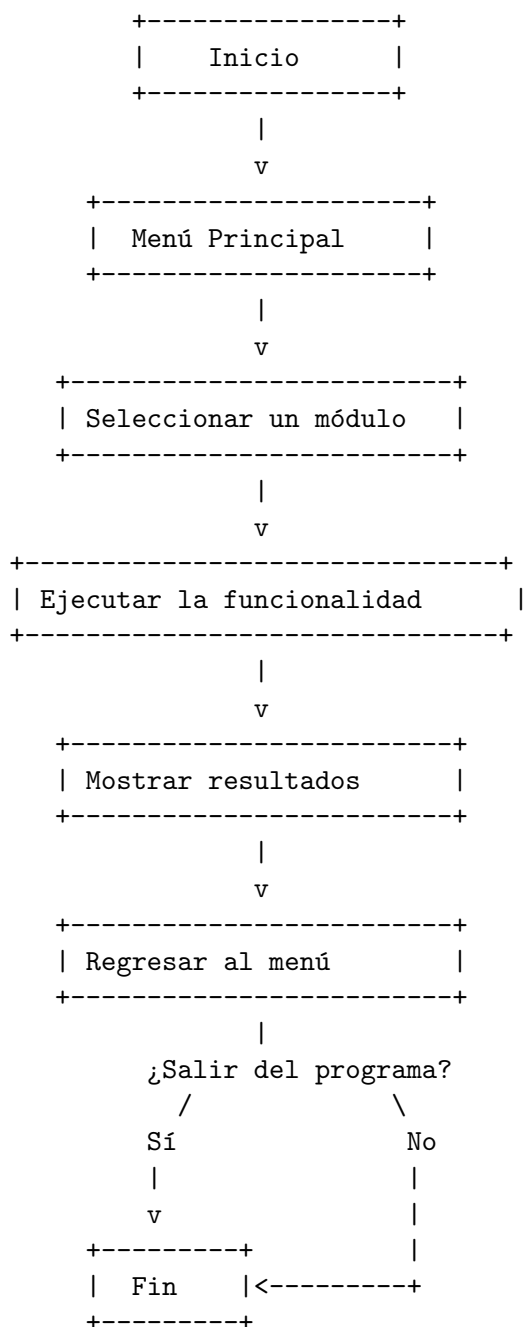


Figura 2. Flujo general de ejecución de la herramienta.

8. Llamadas al sistema y funciones utilizadas

El desarrollo de la herramienta aprovecha diversas funciones de la biblioteca estándar de C y comandos propios del sistema operativo Linux para implementar las funcionalidades solicitadas. Estas funciones permiten interactuar con el sistema operativo, el sistema de archivos y los procesos en ejecución.

8.1. Administración de procesos

Para la gestión de procesos se utilizan comandos propios de Linux invocados desde el programa, permitiendo obtener información y controlar procesos existentes.

Entre las operaciones realizadas se encuentran:

- Consulta de procesos activos.
- Visualización del consumo de CPU y memoria.
- Finalización de procesos.
- Suspensión y reanudación mediante señales del sistema.
- Visualización del árbol de procesos.

Estas funcionalidades permiten supervisar el estado del sistema desde una única interfaz.

8.2. Administración de archivos

Las operaciones sobre archivos utilizan funciones de la biblioteca estándar y comandos del sistema para manipular directorios y archivos.

Las principales operaciones implementadas son:

- Apertura y lectura de directorios.
- Copia de archivos.
- Movimiento de archivos.
- Eliminación de archivos.
- Búsqueda por nombre.
- Obtención de información básica de los archivos.

Estas operaciones permiten administrar el sistema de archivos de manera sencilla desde la aplicación.

8.3. Ejecución de comandos

La herramienta permite ejecutar comandos del sistema operativo ingresados por el usuario.

Para ello se emplean funciones de ejecución de comandos que permiten:

- Enviar el comando al sistema operativo.
- Mostrar la salida generada.
- Registrar el comando ejecutado en un historial.

Esta funcionalidad convierte la aplicación en una interfaz sencilla para la ejecución de comandos Linux.

8.4. Entrada y salida

La interacción con el usuario se implementa mediante las funciones estándar de entrada y salida del lenguaje C.

Las principales funciones utilizadas son:

- `printf()` para mostrar información.
- `scanf()` para capturar datos ingresados por el usuario.
- `fgets()` para leer cadenas de texto completas.
- Funciones de manejo de archivos como `fopen()`, `fprintf()`, `fclose()` y `fgets()` para almacenar historiales, respaldos y registros.

8.5. Manejo de directorios

Para recorrer directorios y acceder a su contenido se emplean funciones proporcionadas por la biblioteca POSIX.

Entre ellas destacan:

- `opendir()`
- `readdir()`
- `closedir()`

Estas funciones permiten listar archivos y obtener información del contenido de un directorio determinado.

8.6. Resumen

La combinación de funciones estándar del lenguaje C, bibliotecas POSIX y comandos propios de Linux permitió implementar una herramienta funcional para la administración del sistema, manteniendo una estructura modular y de fácil mantenimiento.

9. Compilación e instalación del proyecto

El proyecto fue desarrollado para ejecutarse en sistemas operativos Linux, utilizando el compilador GCC y un archivo `Makefile` que automatiza el proceso de compilación.

9.1. Requisitos del sistema

Para compilar y ejecutar la aplicación se requiere:

- Sistema operativo Linux (Ubuntu 22.04 o superior).
- Compilador GCC.
- Utilidad `make`.
- Permisos de lectura y escritura sobre los directorios del proyecto.

Antes de compilar, es recomendable verificar que GCC y Make se encuentren instalados mediante los siguientes comandos:

```
gcc --version
make --version
```

9.2. Estructura del proyecto

El código fuente se encuentra organizado en diferentes directorios para facilitar su mantenimiento.

- `src/`: archivos fuente (.c).
- `include/`: archivos de cabecera (.h).
- `bin/`: ejecutable generado.
- `backups/`: almacenamiento de respaldos.
- `downloads/`: archivos relacionados con las descargas.
- `logs/`: registros de eventos e historial.
- `scripts/`: scripts utilizados para pruebas del analizador Bash.

9.3. Proceso de compilación

La compilación completa del proyecto se realiza ejecutando el siguiente comando desde la carpeta principal del repositorio:

```
make
```

El archivo Makefile compila todos los módulos del sistema y genera el ejecutable principal dentro del directorio correspondiente.

9.4. Limpieza del proyecto

Para eliminar los archivos generados durante la compilación se utiliza el siguiente comando:

```
make clean
```

Esta operación elimina los archivos objeto y el ejecutable generado, permitiendo realizar una compilación limpia cuando sea necesario.

9.5. Ejecución del programa

Una vez compilado el proyecto, la aplicación puede ejecutarse desde la terminal mediante:

```
./bin/admin_linux
```

Al iniciar, el programa muestra el menú principal desde el cual el usuario puede acceder a todas las funcionalidades implementadas.

10. Pruebas realizadas

Con el objetivo de verificar el correcto funcionamiento de la herramienta, se realizaron diferentes pruebas sobre cada uno de los módulos implementados. Estas pruebas permitieron comprobar que las funcionalidades principales operan de acuerdo con los requisitos establecidos para el proyecto.

10.1. Pruebas del administrador de tareas

Se verificó el funcionamiento de las operaciones relacionadas con la administración de procesos del sistema.

Las pruebas realizadas fueron:

- Listado de procesos activos.
- Búsqueda de procesos por nombre.
- Visualización del consumo de CPU y memoria.
- Finalización de procesos seleccionados.
- Suspensión y reanudación de procesos.
- Visualización del árbol de procesos.

Los resultados obtenidos demostraron que el módulo permite consultar y administrar procesos del sistema de manera correcta.

10.2. Pruebas del shell de archivos

Se realizaron pruebas utilizando archivos y directorios de ejemplo.

Las operaciones verificadas fueron:

- Listado del contenido de directorios.
- Copia de archivos.
- Movimiento de archivos.
- Eliminación de archivos.
- Búsqueda de archivos por nombre.
- Consulta de estadísticas de archivos.

Todas las operaciones produjeron los resultados esperados bajo condiciones normales de uso.

10.3. Pruebas de ejecución de comandos

Se ejecutaron distintos comandos propios de Linux para verificar la correcta comunicación entre la aplicación y el sistema operativo.

Entre los comandos probados se encuentran:

- `pwd`
- `ls`
- `whoami`
- `date`

Además, se comprobó que el historial de comandos registra correctamente las ejecuciones realizadas.

10.4. Pruebas del sistema de respaldos

Se verificó la creación y restauración de respaldos utilizando archivos de prueba. Las pruebas permitieron comprobar:

- Creación de nuevas copias de seguridad.
- Almacenamiento organizado de respaldos.
- Restauración de versiones previamente almacenadas.

Los archivos restaurados conservaron correctamente su contenido original.

10.5. Pruebas del analizador Bash

Para validar este módulo se utilizaron diferentes scripts Bash con estructuras variadas. Durante las pruebas se verificó la detección de:

- Variables declaradas.
- Ciclos `for`.
- Ciclos `while`.

Los resultados obtenidos coincidieron con la estructura de los scripts analizados.

10.6. Pruebas de la cola de descargas

Se realizaron pruebas agregando diferentes elementos a la cola de descargas. Se verificó:

- Registro correcto de nuevas tareas.
- Visualización de la cola.
- Registro de eventos.
- Actualización del progreso de las descargas.

El módulo respondió correctamente durante todas las pruebas realizadas.

10.7. Resumen de pruebas

En la Tabla 1 se presenta un resumen de las pruebas ejecutadas sobre cada uno de los módulos del sistema.

Módulo	Resultado
Administrador de tareas	Correcto
Shell de archivos	Correcto
Comandos Linux	Correcto
Respaldos	Correcto
Analizador Bash	Correcto
Cola de descargas	Correcto

Cuadro 1: Resumen de las pruebas funcionales realizadas.

11. Conclusiones y trabajos futuros

11.1. Conclusiones

El desarrollo de la herramienta permitió integrar en una única aplicación diversas funcionalidades de administración para sistemas Linux, cumpliendo con los objetivos planteados al inicio del proyecto. La estructura modular implementada facilitó la organización del código y permitió desarrollar cada componente de forma independiente, favoreciendo el mantenimiento y la escalabilidad del sistema.

Durante la implementación se aplicaron conceptos propios de la programación de sistemas, haciendo uso del lenguaje C, bibliotecas estándar y herramientas del sistema operativo Linux para la gestión de procesos, archivos, ejecución de comandos y administración de recursos.

Cada uno de los módulos desarrollados fue probado de manera independiente, verificando el correcto funcionamiento de las operaciones de administración de procesos, manipulación de archivos, ejecución de comandos, creación y restauración de respaldos, análisis de scripts Bash y gestión de la cola de descargas.

Asimismo, el uso de un archivo Makefile permitió automatizar el proceso de compilación, mientras que la organización del proyecto en módulos independientes contribuyó a mejorar la legibilidad y facilitar futuras modificaciones del código.

En conjunto, el proyecto permitió aplicar de forma práctica los conocimientos adquiridos durante el curso de Programación de Sistemas, integrando diferentes conceptos del sistema operativo Linux en una aplicación funcional.

11.2. Trabajos futuros

Como parte de una posible evolución del proyecto, se plantean las siguientes mejoras:

- Implementar una interfaz gráfica utilizando bibliotecas como GTK o Qt.
- Incorporar monitoreo de procesos en tiempo real con actualización automática de la información.
- Mejorar el sistema de respaldos mediante compresión y programación automática de copias de seguridad.
- Ampliar el analizador de scripts Bash para detectar funciones, condiciones, tuberías y llamadas a otros scripts.
- Agregar soporte para múltiples usuarios y distintos niveles de permisos.
- Implementar un sistema de configuración para personalizar rutas, directorios y preferencias de la aplicación.
- Optimizar el manejo de errores y la validación de entradas para incrementar la robustez del sistema.

Estas mejoras permitirían ampliar las capacidades de la herramienta y acercarla al funcionamiento de aplicaciones de administración utilizadas en entornos Linux reales.